

# Data Engineer (AWS Cloud) — 30-Day Interview Preparation Plan

---

Foundations through advanced topics, structured so you're interview-ready by Day 15.

This guide has two parts: Part 1 is the day-by-day 30-day plan. Part 2 is the full technical reference (SQL, Python/PySpark, every major AWS data service, data modeling, security, performance, scenarios, and behavioral prep) that the plan points you to each day.

## PART 1 — The 30-Day Plan

### How This Is Structured

- Days 1-15 (Weeks 1-2): Foundations plus every core AWS data service you're likely to be asked about. By the end of Day 15 you should be able to hold a competent technical conversation on SQL, Python/PySpark basics, and the AWS data platform — start applying and accepting interviews from here.
- Days 16-22 (Week 3): Advanced depth — Spark internals, streaming architecture, security, performance/cost tuning, governance, CI/CD — the material that separates a competent answer from a strong one, and what typically comes up in second/final rounds.
- Days 23-30 (Week 4): System design practice, mock interviews, behavioral prep, and gap-filling based on real interview feedback from the interviews you'll already be doing by this point.
- Plan for roughly 3-4 focused hours per weekday and a lighter 1-2 hour review on weekends; adjust pacing to your own baseline — if SQL is already strong, compress Days 1-2 and use the saved time for extra Spark/Redshift practice.

### Week 1: Foundations (Days 1-7)

- **Day 1:** SQL fundamentals refresher — joins (inner/left/right/full, self-join), subqueries vs CTEs, GROUP BY/HAVING, and window functions (ROW\_NUMBER, RANK, DENSE\_RANK, LEAD/LAG, running totals with SUM() OVER). Solve 15-20 practice problems (StrataScratch/LeetCode Database style), focusing on window functions since they come up constantly in data engineering interviews.
- **Day 2:** Advanced SQL — recursive CTEs, query execution plans (EXPLAIN/EXPLAIN ANALYZE), indexing strategy (B-tree vs bitmap, when an index is ignored), and query optimization patterns (avoiding SELECT \*, predicate pushdown, avoiding functions on indexed columns in WHERE clauses).
- **Day 3:** Python for data engineering — pandas (DataFrame operations, groupby/agg, merge/join, pivot\_table), reading/writing CSV/JSON/Parquet, and writing a small script

that reads a raw file, cleans it, and writes partitioned Parquet output — this exact pattern shows up in take-home assessments.

- **Day 4:** Data modeling fundamentals — OLTP vs OLAP, normalization (1NF-3NF) vs denormalization trade-offs, dimensional modeling (star vs snowflake schema, fact vs dimension tables), and Slowly Changing Dimensions (Type 1 overwrite, Type 2 historized, Type 3 previous-value column).
- **Day 5:** AWS fundamentals — account/IAM basics (users, roles, policies, least privilege, the trust policy vs permissions policy distinction), VPC basics (subnets, route tables, security groups vs NACLs), and S3 fundamentals (buckets, storage classes, versioning, lifecycle rules).
- **Day 6:** S3 deep dive for data engineering — partitioning key design (Hive-style year=/month=/day= paths), file formats (Parquet/ORC columnar vs CSV/JSON row-based) and why columnar wins for analytics, compression codecs (Snappy vs Gzip trade-offs), and S3 Select for partial-file reads.
- **Day 7:** Review + integration practice — solve 2-3 combined problems that chain SQL + Python + an S3 partitioning decision together; and sketch (on paper) a simple end-to-end pipeline architecture from source system to a queryable table.

## Week 2: Core AWS Data Services (Days 8-15) — Interview-Ready Checkpoint

- **Day 8:** AWS Glue — Glue Data Catalog (the central Hive-metastore-compatible metadata layer), Crawlers (schema inference, classifiers), Glue ETL jobs (Spark-based, DynamicFrame vs DataFrame), Glue Studio visual authoring, job bookmarks for incremental processing, and DPU/worker type sizing (G.1X vs G.2X vs G.025X).
- **Day 9:** Amazon Redshift — architecture (leader node vs compute nodes vs slices), distribution styles (KEY, EVEN, ALL, AUTO) and how a bad distribution key causes data skew, sort keys (compound vs interleaved), Workload Management (WLM) queues, Redshift Spectrum for querying S3 directly, and COPY/UNLOAD for bulk load/export.
- **Day 10:** Amazon Athena — serverless SQL over S3 via Presto/Trino engine, partition projection vs Glue Catalog partitions, CTAS for materializing query results, workgroups for cost/query control, and the per-TB-scanned cost model (which is exactly why partitioning and columnar formats matter here).
- **Day 11:** Amazon EMR — cluster architecture (master/core/task nodes), running Spark/Hive on EMR, EMRFS (S3 as the Hadoop-compatible filesystem), instance fleets vs uniform instance groups, and using Spot instances on task nodes for cost savings without risking data loss.
- **Day 12:** Streaming ingestion — Kinesis Data Streams (shards, partition keys, shard iterators), Kinesis Data Firehose (buffered delivery to S3/Redshift/OpenSearch with format conversion), Kinesis Data Analytics, and how this compares conceptually to self-managed Kafka/MSK.

- **Day 13:** NoSQL and relational AWS databases — DynamoDB (partition key/sort key design, GSI vs LSI, on-demand vs provisioned capacity, DynamoDB Streams for CDC-style triggers) and RDS/Aurora (read replicas, Aurora Serverless v2 for variable workloads, when you'd choose Aurora over standard RDS).
- **Day 14:** Orchestration — AWS Step Functions (state machines, Task/Choice/Parallel states, error handling with Catch/Retry) and Amazon MWAA (Managed Airflow: DAGs, operators, sensors, scheduling, retries, SLA misses) — know when you'd reach for one vs. the other.
- **Day 15:** Build a mini end-to-end project (S3 raw landing -> Glue ETL -> partitioned Parquet -> Redshift or Athena, orchestrated by Step Functions or a simple Airflow DAG), write 3-4 resume-ready bullets describing it, and self-run a mock interview using the Q&A in Part 2. **\*\*This is your checkpoint — start scheduling/attending interviews from here.\*\***

### Week 3: Advanced Depth — Spark, Streaming, Security, Performance (Days 16-22)

- **Day 16:** Spark internals — RDD vs DataFrame vs Dataset APIs, the Catalyst optimizer and Tungsten execution engine, partitioning and shuffling (why a wide transformation like groupBy triggers a shuffle), broadcast joins vs shuffle joins, and caching/persistence (MEMORY\_ONLY vs MEMORY\_AND\_DISK).
- **Day 17:** Streaming architecture patterns — Lambda architecture (separate batch + speed layers) vs Kappa architecture (single streaming layer reprocessed as needed), exactly-once vs at-least-once vs at-most-once delivery semantics, and watermarking for handling late-arriving events in windowed aggregations.
- **Day 18:** Security deep dive — IAM policy structure (identity-based vs resource-based policies, condition keys, the trust policy on a role), KMS encryption (envelope encryption, customer-managed vs AWS-managed keys, encryption at rest vs in transit), Lake Formation fine-grained access control (column/row-level permissions), VPC endpoints (Gateway vs Interface) for private AWS service access, and PII masking/tokenization patterns.
- **Day 19:** Performance and cost optimization — Redshift query tuning (VACUUM/ANALYZE, distribution/sort key redesign, WLM queue tuning), Glue job tuning (DPU count, worker type, partitioning the input, avoiding small-file problems), Athena cost control (partition pruning, columnar formats, compression, workgroup query limits), and S3 cost optimization (Intelligent-Tiering, lifecycle transitions to Glacier).
- **Day 20:** Data governance and quality — AWS Lake Formation's centralized permission model, Glue Data Quality rulesets, data lineage tracking, schema evolution handling (backward/forward compatible schema changes), and validation patterns (contract testing / Great Expectations-style assertions in a pipeline).
- **Day 21:** CI/CD and Infrastructure as Code — provisioning data infrastructure with Terraform or AWS CDK/CloudFormation, deploying Glue jobs/Step Functions via

CodePipeline, and testing strategies for data pipelines (unit tests for transformation logic, data quality checks as a pipeline gate).

- **Day 22:** Review + scenario practice — work through the scenario-based section in Part 2 out loud, and run a second self-mock interview focused specifically on security and performance questions.

#### Week 4: System Design, Mock Interviews & Polish (Days 23-30)

- **Day 23:** System design practice #1 — design a nightly batch ETL pipeline end-to-end for a retail company (sources, ingestion, transformation, storage layers, orchestration, monitoring). Whiteboard it, don't just think about it.
- **Day 24:** System design practice #2 — design a real-time streaming analytics pipeline (e.g., clickstream or IoT sensor data) covering ingestion, processing, storage, and serving layers with latency/throughput trade-offs.
- **Day 25:** System design practice #3 — design a data lake + data warehouse architecture using a medallion pattern (bronze/raw -> silver/cleaned -> gold/aggregated), including where governance and access control fit at each layer.
- **Day 26:** Behavioral interview prep — prepare 4-5 STAR-format stories from real project experience covering: a production incident you resolved, a time you optimized a slow pipeline, a design decision made under ambiguity, and a time you disagreed with a technical approach and how it was resolved.
- **Day 27:** Mock interview #1 (technical, SQL/Python/AWS services) — full run-through with a peer/mentor, or self-recorded and reviewed critically.
- **Day 28:** Mock interview #2 (system design) — full run-through covering one of the three designs from Days 23-25.
- **Day 29:** Gap-filling — revisit whatever came up as weak in your Week 2-3 real interviews (there will be something); this is the single highest-leverage day if you're already interviewing.
- **Day 30:** Final review — flash-review every Q&A section in Part 2, prepare salary/offer negotiation talking points, and draft a thank-you follow-up email template to reuse after interviews.

## PART 2 — Technical Reference (SQL, Python, AWS, Security, Performance, Scenarios)

### 1. SQL Interview Questions

Core concepts to know before the Q&A below:

- SQL splits into DDL (CREATE/ALTER/DROP — schema definition), DML (SELECT/INSERT/UPDATE/DELETE — data manipulation), and DCL (GRANT/REVOKE — permissions); data engineering interviews focus almost entirely on DML, especially SELECT.
- Joins combine rows from two+ tables: INNER (only matching rows), LEFT/RIGHT (all rows from one side plus matches), FULL OUTER (all rows from both sides), and CROSS (Cartesian product) — know how NULLs appear on the non-matching side of an outer join.
- Aggregate functions (SUM, COUNT, AVG, MIN, MAX) collapse multiple rows into one via GROUP BY; HAVING filters after aggregation (WHERE filters before), which is why you can't use an aggregate function inside a WHERE clause.
- Window functions (the OVER clause) compute a value across a set of rows related to the current row without collapsing them — the single most-tested SQL concept in data engineering interviews (ROW\_NUMBER, RANK, LEAD/LAG, running/rolling aggregates).
- Indexes speed up lookups/filters/joins at the cost of slower writes and extra storage; the query planner decides whether to use one based on selectivity and up-to-date statistics.

**Q: Write a query to find the second-highest salary in an Employee table without using LIMIT/OFFSET.**

A: Use a window function: `SELECT DISTINCT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM Employee) t WHERE rnk = 2`. DENSE\_RANK is preferred over RANK here because RANK would skip a rank number after ties (e.g., two people tied for 1st would push the next distinct salary to rank 3, not 2), while DENSE\_RANK correctly treats tied top salaries as a single rank 1 before moving to rank 2.

**Q: How do window functions differ from GROUP BY, and when would you use ROW\_NUMBER vs RANK vs DENSE\_RANK?**

A: GROUP BY collapses rows into one row per group, losing row-level detail; a window function (OVER clause) computes an aggregate or ranking per row while keeping every row in the result set, which is essential when you need both the detail row and a per-group computed value (e.g., each transaction alongside its rank within the customer's transaction history). ROW\_NUMBER assigns a strictly increasing unique number even to ties (arbitrary tie-breaking unless you add a secondary ORDER BY), RANK assigns the same rank to ties but skips subsequent rank numbers, and DENSE\_RANK assigns the same rank to ties without skipping — the right choice depends on whether tied rows should consume a rank slot.

**Q: Explain the difference between a CTE and a subquery, and when a CTE is preferable.**

A: Functionally a CTE (WITH clause) and a derived subquery both produce a temporary named result set scoped to the query, and most modern optimizers (Redshift, PostgreSQL 12+) will inline a non-recursive CTE the same way they'd inline a subquery — so it's not

automatically faster. The real advantage is readability and reuse: a CTE can be referenced multiple times in the same query without repeating the logic, and a recursive CTE (WITH RECURSIVE) can express hierarchical/graph traversal (e.g., an org chart) that a plain subquery cannot express at all.

**Q: How would you find duplicate rows in a table and remove all but one copy?**

A: Identify duplicates with a window function: `SELECT *, ROW_NUMBER() OVER (PARTITION BY col1, col2 ORDER BY id) AS rn FROM table` — rows where `rn > 1` within their partition are the duplicates. To delete them in a database that supports it: `DELETE FROM table WHERE id IN (SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY col1, col2 ORDER BY id) AS rn FROM table) t WHERE rn > 1)`. In a data lake context (Parquet on S3), you'd instead do this as a Spark/Glue transformation — `dropDuplicates(['col1','col2'])` or the windowed approach above expressed in a DataFrame — and rewrite the output rather than mutating files in place.

**Q: What does an execution plan tell you, and what specific things do you look for when diagnosing a slow query?**

A: An execution plan (EXPLAIN, or EXPLAIN ANALYZE for actual runtime stats) shows the physical steps the optimizer chose: scan type (sequential/full scan vs index scan), join algorithm (nested loop vs hash join vs merge join), join order, and estimated vs actual row counts at each step. I look for: a sequential scan on a large table where an index scan was expected (missing or unused index), a nested loop join over large row counts (usually should be a hash join — often caused by a missing statistics update or a data type mismatch preventing index use), and large discrepancies between estimated and actual rows (stale statistics, meaning ANALYZE/VACUUM ANALYZE needs to run).

**Q: How would you write a query to calculate a running 7-day total of daily sales?**

A: Using a window function with a range frame: `SELECT sale_date, daily_total, SUM(daily_total) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7day_total FROM daily_sales ORDER BY sale_date`. `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` gives a 7-row physical window; if the data isn't guaranteed to have exactly one row per day (gaps), a RANGE-based interval window (`RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW`, supported in Postgres/Redshift) is the more correct choice since it respects actual calendar days rather than row count.

## 2. Python & PySpark Interview Questions

Core concepts to know before the Q&A below:

- Pandas is single-machine, in-memory — great for data that fits comfortably in RAM (up to a few GB); it will silently work slowly or crash (MemoryError) once data outgrows the machine.

- PySpark is a distributed processing framework — the same logical operations (filter, join, groupBy) run across many machines in parallel, which is why it's the default choice once data is too large for a single machine or needs to scale with growing volume.
- Spark uses lazy evaluation: transformations (filter, select, join) just build up a execution plan (a DAG) without running anything, and only an action (count, collect, write) actually triggers computation — this is why a 'slow line' in Spark code is often misleading; the real cost shows up at the action.
- A Spark job is broken into stages (separated by shuffle boundaries) and each stage into tasks (one per partition) run in parallel across executors — understanding this hierarchy is essential for reading the Spark UI when diagnosing performance.
- Key libraries: pandas (single-machine dataframes), PySpark (distributed dataframes/SQL), boto3 (AWS SDK for Python, used to call any AWS service programmatically from a script or Lambda).

**Q: How do you read a large CSV file that doesn't fit in memory using pandas?**

A: Use chunksize to iterate over the file in manageable pieces: for chunk in `pd.read_csv('file.csv', chunksize=100000): process(chunk)` — this processes 100K rows at a time instead of loading the full file. For genuinely large-scale data (multi-GB+), pandas isn't the right tool at all — I'd switch to PySpark, which distributes the read across a cluster and only materializes what's needed via lazy evaluation, or use Dask for a pandas-like API with out-of-core/distributed execution.

**Q: What's the difference between a Spark RDD, DataFrame, and Dataset, and why do DataFrames usually perform better?**

A: An RDD is Spark's original low-level distributed collection API — type-safe but opaque to the optimizer, since Spark can't see inside arbitrary lambda functions to optimize them. A DataFrame is a distributed collection organized into named columns (conceptually like a database table) that goes through the Catalyst optimizer and Tungsten's off-heap binary memory format, enabling optimizations like predicate pushdown and whole-stage code generation that RDDs don't get. A Dataset (JVM languages only, not PySpark) adds compile-time type safety on top of the DataFrame execution engine. In practice, DataFrames (or Spark SQL) are the default choice for performance; you'd drop to RDD only for very custom, low-level operations the DataFrame API can't express.

**Q: Explain what causes a shuffle in Spark, and how you'd reduce shuffle cost in a job.**

A: A shuffle happens when data needs to be redistributed across partitions because a transformation requires rows with the same key to be co-located — `groupBy`, `join` (unless broadcast), `distinct`, and `repartition` all trigger one, and it involves writing intermediate data to disk and transferring it over the network between executors, which is typically the most expensive part of a Spark job. To reduce shuffle cost: use broadcast joins (`broadcast(smallDF)`) when one side of a join is small enough to fit in executor memory (avoids shuffling the large side entirely), pre-partition data by the join/group key at write time so downstream shuffles are smaller, increase `spark.sql.shuffle.partitions` from its

default of 200 if partitions are too large (causing spill to disk) or decrease it if there are too many tiny partitions (task scheduling overhead), and use `reduceByKey` instead of `groupByKey` on RDDs since `reduceByKey` combines values locally before shuffling.

**Q: How would you handle schema evolution when reading Parquet files that were written at different times with slightly different schemas?**

A: Spark's Parquet reader supports schema merging via `mergeSchema=true` (or `spark.sql.parquet.mergeSchema=true`), which unions the schemas across files and fills missing columns with null — but this has a real performance cost since Spark must read the footer of every file to build the merged schema, so it's not something to leave on by default for a large table. A more controlled approach is to explicitly define and enforce a target schema (`schema=my_schema` when reading), using an explicit cast/select to align older files to the current schema, and to design new columns as nullable/additive from the start so old readers/writers don't break — this is the same additive-schema-evolution discipline used in Avro/Protobuf-based systems.

**Q: Write a PySpark snippet to deduplicate records keeping the most recent version per key, and explain the performance consideration.**

A:

- `from pyspark.sql import Window`
- `from pyspark.sql.functions import row_number, col`
- 
- `w = Window.partitionBy("customer_id").orderBy(col("updated_at").desc())`
- `deduped_df = (df.withColumn("rn", row_number().over(w))`
- `.filter(col("rn") == 1)`
- `.drop("rn"))`
- This pattern requires a shuffle (`partitionBy` triggers a repartition by `customer_id`), so for a very large dataset it's worth first checking whether the source can be pre-filtered (e.g., only recent partitions) or whether an incremental/upsert pattern (writing only changed records via a merge into a table format like Delta/Iceberg/Hudi) would avoid re-deduplicating the entire historical dataset on every run.

### 3. AWS Storage & Compute for Data Engineering

Core concepts to know before the Q&A below:

- S3 (Simple Storage Service) is object storage — not a filesystem or database — and is the de facto foundation of almost every AWS data lake, since every major AWS analytics service (Athena, Redshift Spectrum, EMR, Glue) can read directly from it.
- An S3 'folder' is just a naming convention (a common key prefix, e.g., `raw/2024/01/15/file.parquet`) — there's no real directory structure underneath, which matters when reasoning about partitioning and listing performance.
- Lambda is serverless compute — you supply code, AWS runs it on-demand in response to a trigger (an S3 event, an API call, a schedule) and you pay only for execution time, with no server to manage but a hard 15-minute execution ceiling.
- EC2 is raw virtual machines you manage yourself (OS, patching, scaling) — rarely the direct tool of choice for data engineering pipelines today, but it's what underlies managed services like EMR and Redshift under the hood.
- IAM roles (not long-lived access keys) are the standard way AWS services authenticate to each other — e.g., a Glue job 'assumes' an IAM role that grants it permission to read a specific S3 bucket, rather than a hardcoded username/password.

**Q: Walk through the S3 storage classes and how you'd design a lifecycle policy for a data lake.**

A: S3 Standard for frequently accessed hot data, S3 Standard-IA/One Zone-IA for infrequently accessed data still needing millisecond retrieval, S3 Intelligent-Tiering for unpredictable access patterns (automatically moves objects between tiers based on observed access), and S3 Glacier Instant/Flexible/Deep Archive for cold data with retrieval times ranging from milliseconds to 12+ hours. A typical data lake lifecycle policy: keep raw/bronze data in Standard for 30 days (active reprocessing window), transition to Standard-IA at 30 days, transition to Glacier Flexible Retrieval at 90-180 days for long-term retention/compliance, and set an expiration rule if there's a defined retention limit — configured via an S3 Lifecycle Configuration with prefix/tag-based rules so different data domains can have different policies within the same bucket.

**Q: How do you design an S3 partitioning scheme for a table that will be queried by Athena/Redshift Spectrum, and what's the small-files problem?**

A: Use Hive-style partitioning in the object key path (`s3://bucket/table/year=2024/month=01/day=15/file.parquet`) aligned to the columns most commonly filtered in WHERE clauses (typically date, since most analytical queries filter by a date range) — the query engine then only reads the specific partition prefixes matching the filter instead of scanning the whole table (partition pruning). The small-files problem occurs when a streaming or micro-batch process writes many small files per partition (e.g., thousands of KB-sized files from Kinesis Firehose); this hurts performance because the query engine pays per-file overhead (listing, opening, reading footers) that dominates over actual data scanning — mitigated by a periodic compaction job (e.g., a Glue job that reads a partition and rewrites it as fewer, larger files, targeting roughly 128MB-1GB per file) or by tuning Firehose's buffering interval/size to produce fewer, larger files at ingestion time.

**Q: What's the difference between AWS Lambda and Glue for running a data transformation, and when would you choose each?**

A: Lambda has a 15-minute maximum execution time and limited memory (up to 10GB), making it suited to lightweight, event-driven transformations (e.g., reacting to an S3 PUT event to validate or lightly reshape a single file) but unsuitable for processing large datasets that need distributed compute. Glue runs Spark under the hood, so it's built for genuinely large-scale, distributed ETL — joining large datasets, aggregating across millions of rows, or handling data volumes that don't fit on a single machine — at the cost of higher per-job startup latency (cold start for provisioning DPUs) and cost that doesn't make sense for tiny, frequent triggers. As a rule of thumb: Lambda for small, fast, event-driven work; Glue for anything that needs Spark's distributed processing.

#### **4. AWS Glue, Step Functions & Orchestration**

Core concepts to know before the Q&A below:

- Glue is AWS's managed ETL service: it runs Apache Spark under the hood (so everything you know about Spark applies), but AWS provisions and manages the cluster for you per-job rather than you standing up EMR yourself.
- ETL = Extract, Transform, Load (transform happens before loading to the destination); ELT = Extract, Load, Transform (raw data lands first, transformation happens afterward, often inside the warehouse) — modern cloud data platforms lean ELT because storage is cheap and warehouses like Redshift/Snowflake have strong in-database transform capability.
- Orchestration means controlling the order, timing, retries, and dependencies between pipeline steps that might otherwise be independent scripts — without it you have a collection of jobs, not a reliable pipeline.
- A DAG (Directed Acyclic Graph) is the standard way to represent a pipeline's dependencies — 'acyclic' meaning no step can depend on itself even indirectly, which both Airflow and Step Functions enforce structurally.
- Idempotency (safe to re-run without side effects) is a design property you build into every pipeline step deliberately — it's what makes retries after a failure safe rather than dangerous (e.g., risking duplicate data).

**Q: What is the AWS Glue Data Catalog, and how does it relate to Athena, Redshift Spectrum, and EMR?**

A: The Glue Data Catalog is a centralized, Hive-metastore-compatible metadata repository storing table definitions (schema, location, partition info, file format) without storing the actual data. Athena, Redshift Spectrum, and EMR (via Hive/Spark) can all point at the same Glue Catalog to query the same underlying S3 data with a consistent schema definition, which avoids each engine maintaining its own separate metadata and means a Crawler run once updates the schema visibility for all three consumers simultaneously.

**Q: What's a Glue job bookmark, and what problem does it solve?**

A: A job bookmark tracks which source data (e.g., which S3 objects, or which primary key range for a JDBC source) has already been processed by a Glue job across runs, so a subsequent run only processes new/changed data instead of reprocessing the entire source every time — critical for incremental ETL. It's enabled via `job.init()` with the job name and the `--job-bookmark-option enable` parameter, and Glue persists the bookmark state internally between runs; a common gotcha is that bookmarks are keyed to the specific job name and transformation context, so renaming a job or changing the underlying script's structure can reset or break bookmark tracking, requiring a manual bookmark reset if that happens unintentionally.

**Q: How do you decide between AWS Step Functions and Amazon MWAA (Managed Airflow) for orchestrating a pipeline?**

A: Step Functions is a native AWS state machine service, well-suited for orchestrating primarily AWS-service-to-AWS-service workflows (Lambda, Glue, EMR, ECS) with visual state tracking, built-in retry/Catch error handling per state, and pay-per-state-transition pricing that's cost-effective for less frequent or bursty workflows. MWAA (managed Apache Airflow) is the better fit when you need complex DAG dependencies, a large ecosystem of pre-built operators/hooks for both AWS and non-AWS systems, dynamic DAG generation, or your team already has Airflow expertise/existing DAGs to migrate — but it runs on persistent infrastructure (an Airflow environment), so it has a baseline cost even when idle, unlike Step Functions' pure pay-per-use model. In practice, many architectures use both: MWAA for overall pipeline scheduling/dependency management, calling into Step Functions for specific AWS-native sub-workflows.

**Q: How would you handle a failure partway through a multi-step Step Functions state machine (e.g., Glue job succeeds, but the downstream Redshift load fails)?**

A: Configure a Catch block on the failing state to route to a dedicated error-handling state (e.g., an SNS notification plus writing failure details to a DynamoDB table or CloudWatch Logs for triage), combined with a Retry block using exponential backoff (`IntervalSeconds`, `BackoffRate`, `MaxAttempts`) for transient failures like a temporary Redshift connection issue. For the specific case of a partial pipeline failure, I'd design the state machine so completed steps are idempotent/re-runnable (e.g., the Redshift load uses a MERGE/UPSERT pattern keyed on a natural key, not a blind INSERT) so that re-triggering the execution from the failed state — or the whole execution — doesn't create duplicate data.

## 5. Amazon Redshift Deep Dive

Core concepts to know before the Q&A below:

- Redshift is AWS's managed data warehouse, built on MPP (Massively Parallel Processing) architecture — a single query is split across multiple compute nodes that each work on a slice of the data in parallel, which is fundamentally different from a single-machine OLTP database like standard RDS.

- Redshift is columnar storage (like Parquet) rather than row storage — it stores each column's values together on disk, which makes analytical queries that scan a few columns across millions of rows much faster than a row-store would be, at the cost of being poor for single-row lookups/updates.
- A cluster has one leader node (receives queries, builds the execution plan, coordinates compute nodes) and multiple compute nodes (each further divided into slices that store and process a portion of the data) — this leader/compute split is the physical basis for everything about distribution keys and query parallelism.
- Redshift is OLAP (Online Analytical Processing) — optimized for complex read-heavy aggregation queries over large data — as opposed to OLTP (Online Transaction Processing) systems like RDS/Aurora, optimized for many small, fast read/write transactions.

**Q: Explain Redshift's distribution styles and how a poor choice causes performance problems.**

A: KEY distribution hashes rows to compute nodes based on a specified column's value, colocating rows with the same key on the same node — ideal for large fact tables joined frequently on that key, since the join can happen locally without shuffling data across nodes. EVEN distribution round-robins rows evenly regardless of value, giving balanced storage but requiring a network shuffle for most joins. ALL distribution replicates the full table to every node, appropriate only for small, frequently-joined dimension tables (replicating a large table this way wastes storage and slows writes). A poor choice — e.g., EVEN distribution on a huge fact table that's constantly joined to another huge table on a specific key — forces expensive cross-node data redistribution on every query, which shows up in EXPLAIN plans as a DS\_BCAST\_INNER or DS\_DIST\_BOTH step and is usually the single biggest lever for fixing a slow Redshift query.

**Q: What's the difference between compound and interleaved sort keys, and when would you use each?**

A: A compound sort key physically orders data by the listed columns in sequence (like a multi-column index), giving excellent performance for queries that filter/sort on a prefix of those columns (especially the first one) but degrading for queries that filter primarily on later columns in the key. An interleaved sort key gives equal weight to all specified columns, performing well for queries that filter on any of several different columns unpredictably — at the cost of slower VACUUM REINDEX operations and slower load performance compared to a compound key. In practice, compound keys (often just the date column, since most analytical queries filter by date range) are the more common and lower-maintenance default; interleaved keys are reserved for genuinely multi-dimensional filter patterns where no single column dominates.

**Q: Why does Redshift require regular VACUUM and ANALYZE, and what happens if you skip them?**

A: Redshift stores data in sorted, compressed blocks; DELETE and UPDATE operations don't immediately reclaim space or re-sort data — they mark rows as deleted (ghost rows) and

append new rows unsorted at the end of the table. VACUUM reclaims that space and re-sorts the table according to its sort key; without it, storage bloats with ghost rows and query performance degrades because more unsorted blocks must be scanned, defeating the purpose of the sort key entirely. ANALYZE updates table statistics (row counts, distribution) that the query planner relies on to choose join order and join algorithm; stale statistics after significant data changes can cause the optimizer to pick a bad plan (e.g., a nested loop where a hash join would be far faster). Auto-vacuum and auto-analyze are enabled by default in newer Redshift, but for very large tables or predictable bulk-load patterns, I'd still schedule explicit VACUUM/ANALYZE after major loads rather than relying solely on the automatic background process's timing.

**Q: How does Redshift Spectrum differ from loading data into Redshift, and when would you use it?**

A: Redshift Spectrum queries data directly in S3 (via the Glue Data Catalog for schema) without loading it into Redshift's local storage, using a separate fleet of Spectrum compute nodes that scale independently of your cluster. It's the right choice for querying large historical/cold data that's queried infrequently (avoiding the storage cost and load time of bringing it into the cluster) or for joining local Redshift tables against a much larger S3-resident dataset — but it's slower per-query than native Redshift storage since it involves scanning S3 and network transfer, and you pay per-TB-scanned similar to Athena, so the same partitioning/columnar-format optimizations that apply to Athena apply here too.

## 6. Amazon Athena & EMR

Core concepts to know before the Q&A below:

- Athena is a serverless query engine (built on Presto/Trino) that runs SQL directly against files sitting in S3 — there's no cluster to provision or manage, and you pay per query based on data scanned, not for idle infrastructure.
- EMR (Elastic MapReduce) is AWS's managed Hadoop/Spark cluster service — unlike Athena, you provision an actual cluster (with a choice of instance types and size) that stays running for as long as you need it, giving more control at the cost of more to manage.
- The practical distinction: reach for Athena for ad hoc or lighter, infrequent SQL queries over data already in S3; reach for EMR when you need heavier, sustained distributed processing (large Spark jobs, custom Hadoop ecosystem tools) or long-running interactive clusters for a data science team.
- Both ultimately rely on the same Glue Data Catalog for table/schema metadata when configured to do so, meaning the same table definition can serve queries from Athena, Redshift Spectrum, and EMR without redefining it three times.

**Q: Athena charges per TB scanned — what specific techniques reduce both cost and query latency?**

A: Partition the data (and ensure the query's WHERE clause actually filters on the partition column so partition pruning applies — filtering on a derived/transformed version of the partition column defeats pruning), store data in a columnar format (Parquet or ORC) so Athena only reads the specific columns referenced in the query instead of full rows, apply compression (Snappy is a good default — splittable and fast to decompress), avoid the small-files problem (many tiny files add per-file overhead disproportionate to the data volume), and use CTAS to pre-aggregate/pre-join frequently-queried combinations into a materialized table rather than repeating an expensive join on every query.

**Q: What's the difference between an EMR instance fleet and a uniform instance group, and why would you use Spot instances on task nodes but not core nodes?**

A: A uniform instance group runs a single instance type per node role (master/core/task); an instance fleet lets you specify multiple instance types and provisioning options (On-Demand and Spot mixed) per role, letting EMR provision whichever available capacity best matches your target, which improves Spot availability and resilience to a single instance type being capacity-constrained. Core nodes host HDFS data blocks (if HDFS is used) and run both compute and storage, so losing a core node to a Spot interruption risks data loss/rebalancing overhead; task nodes are compute-only (no HDFS storage responsibility), so a Spot interruption there just loses in-progress compute for that node's tasks, which Spark/YARN will simply reschedule elsewhere — making task nodes the safe place to use Spot for cost savings without risking data durability.

## 7. AWS Streaming: Kinesis & Real-Time Architecture

Core concepts to know before the Q&A below:

- Batch processing runs on a schedule over a bounded chunk of data (e.g., 'process yesterday's orders every night at 2am'); stream processing runs continuously over unbounded data as it arrives, aiming for results within seconds rather than hours.
- Kinesis is AWS's managed streaming service (broadly analogous to Apache Kafka) — producers write records to a stream, consumers read and process them, with the stream retaining data for a configurable window (24 hours to 365 days) so multiple consumers can read independently at their own pace.
- Event time (when something actually happened) vs. processing time (when your system received/processed it) is a foundational streaming concept — network delays and retries mean these are rarely identical, and correct windowed aggregations must be computed on event time, not processing time.
- Windowing groups streaming events into time buckets (tumbling/fixed, sliding, or session windows) so you can compute aggregates like 'orders per 5-minute window' on an otherwise never-ending stream of data.

**Q: Explain how Kinesis Data Streams shards work, and how you'd design a partition key strategy to avoid a 'hot shard.'**

A: A Kinesis stream is divided into shards, each with a fixed throughput limit (1MB/sec or 1,000 records/sec ingest per shard); the partition key you supply on each record is hashed to deterministically map it to a specific shard, meaning records with the same partition key always land on the same shard (important for maintaining order per key). A hot shard occurs when the partition key distribution is skewed — e.g., using a low-cardinality key like 'region' when 80% of traffic is from one region overloads that region's shard while others sit idle. The fix is choosing a higher-cardinality, more evenly distributed partition key (e.g., a composite of user ID and a random suffix, or just user ID if user volume is naturally well-distributed) and monitoring the IncomingBytes/IncomingRecords CloudWatch metrics per shard to detect skew, then resharding (split/merge) if needed.

**Q: What's the difference between Kinesis Data Streams and Kinesis Data Firehose, and when would you use each?**

A: Data Streams is a low-latency (sub-second), consumer-managed stream where you write custom consumer application code (KCL or Lambda) to process records as they arrive, giving full control over processing logic and supporting multiple parallel consumers reading the same stream independently. Firehose is a fully managed delivery service with no custom consumer to write — you configure a destination (S3, Redshift, OpenSearch, an HTTP endpoint) and Firehose buffers and batches records (configurable by size or time interval) before delivering, optionally with built-in format conversion (e.g., JSON to Parquet) and compression. Use Data Streams when you need real-time custom processing (e.g., real-time fraud scoring); use Firehose when you just need reliable, low-effort delivery of streaming data into a storage/analytics destination without writing consumer code.

**Q: How do you handle late-arriving or out-of-order events in a streaming aggregation (e.g., hourly counts)?**

A: Use event-time processing with watermarking: the stream processor (Spark Structured Streaming, Kinesis Data Analytics/Flink) tracks the maximum event timestamp seen so far and defines a watermark (e.g., 'allow events up to 10 minutes late') that determines how long a windowed aggregation stays open before being finalized and emitted. Events arriving within the watermark threshold get correctly included in their event-time window even if they arrive out of order; events arriving after the watermark has passed are either dropped or routed to a separate late-data handling path (e.g., a correction/backfill record merged in a subsequent batch job), which is a deliberate trade-off between result completeness and how long you're willing to delay emitting results.

## **8. DynamoDB, RDS & Aurora**

Core concepts to know before the Q&A below:

- DynamoDB is a fully managed NoSQL key-value/document database — no fixed schema across items, automatic horizontal scaling, and single-digit-millisecond latency at virtually any scale, in exchange for much more limited query flexibility than SQL (you generally design tables around your known access patterns up front, not ad hoc joins later).

- RDS is managed relational database hosting (MySQL, PostgreSQL, SQL Server, Oracle, MariaDB) — AWS handles patching/backups/failover, but it's still a traditional single-primary relational engine underneath.
- Aurora is AWS's own cloud-native relational engine (MySQL/PostgreSQL-compatible wire protocol) with a re-architected distributed storage layer, giving higher throughput and faster replication/failover than standard RDS running the same engine.
- The core NoSQL vs. relational decision: relational (RDS/Aurora) when you need complex multi-table joins, strong transactional (ACID) guarantees across multiple entities, and flexible ad hoc querying; NoSQL (DynamoDB) when you have a well-known, simple access pattern and need to scale to very high, predictable-latency throughput.

**Q: How do you design a DynamoDB partition key and sort key to avoid hot partitions, using a concrete example?**

A: DynamoDB distributes data across partitions based on the hash of the partition key, and throughput is provisioned/allocated per partition, so a low-cardinality or skewed partition key (e.g., using 'order\_status' with values like PENDING/COMPLETE as the partition key) concentrates traffic on very few partitions and throttles under load. For an orders table, a better design uses customer\_id as the partition key (naturally high cardinality, spreading load across many customers) and order\_date or order\_id as the sort key, enabling efficient range queries like 'all orders for this customer in the last 30 days' via a sort key condition, while a Global Secondary Index (GSI) on order\_status plus a date attribute supports the operational query pattern of 'all pending orders' without concentrating the base table's partition key on status.

**Q: What's the difference between a DynamoDB Global Secondary Index (GSI) and a Local Secondary Index (LSI)?**

A: An LSI shares the same partition key as the base table but a different sort key, must be created at table creation time (cannot be added later), and shares the base table's read/write capacity and 10GB-per-partition-key-value size limit. A GSI can have a completely different partition key and sort key from the base table, can be added or removed at any time after table creation, and has its own independently provisioned capacity — making it far more flexible for supporting additional query patterns discovered after the table is already in production, which is why GSIs are the more commonly used option in practice despite LSIs having marginally stronger consistency guarantee options.

**Q: When would you choose Aurora over standard RDS, and what is Aurora Serverless v2 useful for?**

A: Aurora is AWS's cloud-native relational engine (MySQL- and PostgreSQL-compatible) that decouples compute from a distributed, self-healing storage layer replicated across 3 Availability Zones automatically, giving higher throughput, faster failover (typically under 30 seconds), and up to 15 read replicas with minimal replication lag compared to standard RDS's traditional replication — at a higher cost per hour than equivalent standard RDS. Aurora Serverless v2 scales compute capacity (in fine-grained ACU increments) up and down automatically based on load without the connection-dropping 'cold start' pause that

Serverless v1 had, making it well-suited for workloads with unpredictable or spiky traffic (e.g., a dev/test environment, or an application with strong daytime/nighttime usage variance) where paying for constantly-provisioned peak capacity would be wasteful.

## 9. Data Modeling, Data Lake & Governance

Core concepts to know before the Q&A below:

- Dimensional modeling organizes a warehouse into fact tables (measurable events — an order, a transaction, with numeric measures and foreign keys) and dimension tables (descriptive context — customer, product, date), and a star schema is simply one fact table surrounded directly by its dimension tables.
- A snowflake schema further normalizes dimension tables into sub-dimensions (e.g., splitting a Product dimension into Product and Category tables) — this saves some storage/redundancy at the cost of requiring more joins per query, which is why star schemas are generally preferred for pure query performance.
- A data lake stores raw and semi-structured data cheaply at scale (typically on S3) without requiring an upfront schema; a data warehouse stores structured, modeled, query-optimized data — a modern 'lakehouse' pattern blends both, applying warehouse-like structure and governance on top of lake storage.
- Data governance covers who can access what data, how data quality is enforced, and how data lineage (where did this value originate, what transformed it) is tracked — increasingly a first-class engineering concern, not just a compliance afterthought.

### **Q: Walk through implementing a Slowly Changing Dimension Type 2 in a data warehouse.**

A: Type 2 preserves history by inserting a new row for every change rather than overwriting, so the dimension table needs additional metadata columns: a surrogate key (independent of the natural/business key, since the same business entity now has multiple rows), `effective_start_date`, `effective_end_date`, and an `is_current` flag. On a change, the ETL process: (1) expires the current row by setting its `effective_end_date` to the change date and `is_current` to false, (2) inserts a new row with the updated attribute values, a new surrogate key, `effective_start_date` set to the change date, `effective_end_date` set to a far-future sentinel date (e.g., 9999-12-31), and `is_current` true. Fact table records loaded going forward reference the new surrogate key, while historical fact records keep referencing the old surrogate key — which is exactly what preserves the ability to report 'what did this customer's address look like at the time of this historical order,' the entire point of Type 2.

### **Q: What's the medallion architecture (bronze/silver/gold), and what's the technical purpose of each layer beyond just naming convention?**

A: Bronze is the raw, unmodified landing zone (data as received from source, append-only, minimal/no transformation) that serves as the immutable source of truth for reprocessing if downstream logic changes or a bug is found — you never want to lose the ability to replay from bronze. Silver applies cleansing, deduplication, type enforcement, and conformance to

a defined schema (still generally at a similar grain to source, not yet business-aggregated), making it the layer most data engineers and data scientists work against for ad hoc analysis. Gold contains business-level aggregates and dimensional models (star schemas, pre-joined/pre-aggregated tables) optimized for specific consumption patterns like BI dashboards, where query performance and business-friendly naming matter more than raw fidelity. The technical purpose of separating these isn't just organizational — it lets you reprocess silver/gold from bronze without re-ingesting from source systems, and it isolates the blast radius of a transformation bug to the layer it was introduced in.

**Q: How does AWS Lake Formation provide governance beyond what S3 bucket policies and IAM alone can do?**

A: S3 bucket policies and IAM grant access at the bucket/prefix level, which is coarse — anyone with read access to a prefix sees every column and row in every file there. Lake Formation adds a centralized permissions layer on top of the Glue Data Catalog that supports column-level and row-level (via data filters) access control, so two different roles can query the same underlying table but see different columns (e.g., masking a salary column) or different row subsets (e.g., a regional manager only sees their region's rows) — enforced consistently whether the access comes through Athena, Redshift Spectrum, or EMR, since all three respect Lake Formation permissions when configured to use it, rather than each engine needing its own separate access control layer.

## 10. Security

Core concepts to know before the Q&A below:

- AWS's Shared Responsibility Model: AWS secures the underlying infrastructure ('security of the cloud'); you're responsible for how you configure IAM, encryption, network access, and data classification ('security in the cloud') — nearly every security question in a data engineering interview is really testing your grasp of your side of that line.
- IAM (Identity and Access Management) is the permission system for everything in AWS: policies (JSON documents listing allowed/denied actions on resources) are attached to users, groups, or roles, and roles (rather than long-lived user credentials) are the standard way services and applications authenticate.
- Encryption at rest protects stored data (e.g., S3 objects, Redshift/RDS storage), encryption in transit protects data moving over the network (TLS/SSL) — a mature data platform applies both, and KMS (Key Management Service) is AWS's managed service for creating and controlling the encryption keys behind either.
- The principle of least privilege means every identity (user, role, service) should have only the specific permissions it needs to do its job, nothing more — the single most common theme across real-world AWS security incidents is a role or key with far broader access than it actually used.

**Q: Explain the difference between an IAM identity-based policy and a resource-based policy, and give an example of when you need both.**

A: An identity-based policy is attached to a user, group, or role and defines what actions that identity can perform on which resources. A resource-based policy is attached directly to a resource (e.g., an S3 bucket policy, a KMS key policy, a Lake Formation resource share) and defines who can access that specific resource, which is essential for cross-account access — the accessing principal's identity-based policy alone isn't sufficient if the resource lives in a different AWS account, because the resource owner's account must also explicitly grant access via its resource-based policy. A cross-account S3 data-sharing scenario needs both: the consuming account's IAM role needs an identity-based policy allowing `s3:GetObject`, and the producing account's bucket policy needs a matching statement granting that specific role/account access — missing either side results in an Access Denied even though the other side looks correctly configured.

**Q: How does S3 server-side encryption with KMS (SSE-KMS) work, and what's 'envelope encryption'?**

A: Envelope encryption means your actual object data is encrypted with a data encryption key (DEK), and that DEK itself is encrypted by a KMS-managed key encryption key (KEK) — the encrypted DEK is stored alongside the object, but the KEK never leaves KMS. When you PUT an object with SSE-KMS, S3 calls KMS to generate a unique DEK per object, encrypts the object with it, then discards the plaintext DEK and stores only the KMS-encrypted version; on GET, S3 calls KMS to decrypt that stored DEK (which requires the caller to have `kms:Decrypt` permission on the key, providing a second access-control layer beyond just S3 permissions) and uses it to decrypt the object. This is why revoking a principal's KMS key permissions effectively revokes their ability to read the data even if their S3 permissions are unchanged — a useful defense-in-depth control.

**Q: What's the difference between a VPC Gateway endpoint and a VPC Interface endpoint, and which would you use for S3 vs. other services?**

A: A Gateway endpoint is a route-table target (no ENI, no additional cost) supported only for S3 and DynamoDB, routing traffic to those services privately without traversing the public internet or requiring a NAT gateway — the standard, low-cost choice for private S3/DynamoDB access from a VPC. An Interface endpoint provisions an Elastic Network Interface (ENI) with a private IP in your subnet, backed by AWS PrivateLink, supporting most other AWS services (Glue, Kinesis, Athena, Secrets Manager, etc.) and incurring an hourly + per-GB cost per endpoint. In a data pipeline running in a private subnet with no internet access, you'd typically combine an S3 Gateway endpoint (free) with Interface endpoints for whichever other services (Glue, Secrets Manager for DB credentials) the pipeline needs to reach privately.

**Q: How would you implement PII masking for a column that most users shouldn't see in plaintext, but a compliance team occasionally needs full access to?**

A: The cleanest approach uses Lake Formation column-level permissions or a dedicated masked view: create a view/table where the PII column is tokenized or hashed (e.g., a one-way hash for values only used for joins/grouping, or format-preserving encryption if the

original value must occasionally be recoverable) for general access, and grant the compliance team a separate, tightly audited role with permission to query the underlying unmasked table or a reversible-decryption function backed by a restricted KMS key. Every access to the unmasked data should be logged (CloudTrail data events on the table/key, or application-level audit logging) so there's a clear record of who accessed unmasked PII and when, which is typically a compliance requirement in its own right (e.g., under GDPR/HIPAA-style regimes) independent of the masking mechanism itself.

**Q: What's the principle of least privilege in practice for a Glue job's IAM role, and what's a common mistake?**

A: In practice, least privilege means the Glue job's execution role should only have `s3:GetObject/PutObject` scoped to the specific bucket/prefix paths it actually reads from and writes to (not `s3:*` on all buckets), `glue:GetTable/UpdateTable` scoped to the specific database/tables it interacts with in the Data Catalog, and no broader administrative permissions even though a Data Engineer building the job might have those personally. A common mistake is attaching the AWS-managed `AWSGlueServiceRole` or a broad `AdministratorAccess`-style policy directly to the job's execution role during development for convenience and never tightening it before production — this means a bug or a compromised job script has far more blast radius (every bucket the account owns) than the job actually needs, which is exactly the kind of gap a security review should catch before go-live.

## 11. Performance & Cost Optimization

Core concepts to know before the Q&A below:

- Most AWS data services charge along one of a few axes: compute time provisioned (EMR, Redshift provisioned clusters), data scanned per query (Athena, Redshift Spectrum), or storage volume plus request count (S3, DynamoDB) — knowing which axis a service bills on tells you exactly which lever actually reduces cost for it.
- Performance tuning in distributed data systems almost always comes back to one of a small set of root causes: data skew (uneven work distribution), too much data movement (shuffles, cross-node joins, scanning unfiltered data), or under/over-provisioned parallelism (too few or too many workers/partitions for the actual data volume).
- Partition pruning (skipping irrelevant partitions entirely) and predicate/column pushdown (letting the storage format skip irrelevant data or columns before reading) are the two most impactful optimizations across Athena, Redshift Spectrum, and Spark alike — most performance and cost problems in this space trace back to one of these not happening as expected.
- Cost and performance are often but not always aligned — sometimes the fastest option (e.g., an always-on provisioned Redshift cluster) costs more than a slower, cheaper serverless alternative (Athena), so the right answer depends on the actual query frequency/latency requirement, not just raw speed.

**Q: A Glue ETL job that used to run in 10 minutes now takes over an hour after the source data grew. Walk through your diagnostic approach.**

A: First check the Glue job's Spark UI/CloudWatch metrics for the actual bottleneck: data skew (a few tasks taking far longer than others, visible as a long tail in the task duration distribution — usually caused by a skewed join or group-by key), excessive shuffle/spill to disk (visible in the Spark UI's shuffle read/write and spill metrics), or simply insufficient parallelism (too few DPUs/workers for the new data volume, or too few input partitions limiting how much work can run concurrently). Fixes depend on the diagnosis: for skew, salting the skewed key or isolating and handling the skewed values separately; for insufficient parallelism, increasing the number of workers/DPUs or repartitioning the input; for a shuffle-heavy join, switching to a broadcast join if one side is now small enough, or ensuring both sides are reasonably partitioned beforehand. I'd also check whether the small-files problem crept in on the source side as data grew, since that degrades read performance independent of the transformation logic itself.

**Q: How would you reduce Athena query cost for a team that frequently reruns similar large aggregation queries?**

A: First, confirm the underlying table is actually partitioned and in a columnar format — if not, that's the highest-leverage fix. Beyond that: use CTAS to materialize the common aggregation as its own smaller table refreshed on a schedule (e.g., nightly), so ad hoc dashboard queries hit the small pre-aggregated table instead of re-scanning the full raw dataset on every dashboard refresh; set per-query and per-workgroup data scan limits in Athena workgroups to catch runaway/unfiltered queries before they run up cost; and if the same exact query genuinely repeats often with identical parameters, Athena's query result reuse feature (if enabled) can serve a cached result within a configurable freshness window instead of re-scanning.

**Q: What's the trade-off between over-partitioning and under-partitioning a table, and how do you decide on partition granularity?**

A: Under-partitioning (e.g., partitioning only by year for a table queried mostly by day) means queries scan far more data than needed since partition pruning can only eliminate whole years, not days within them. Over-partitioning (e.g., partitioning by minute for a moderately-sized table) creates a huge number of small partitions, each potentially containing very little data — this both causes the small-files problem and adds metadata overhead (the Glue Catalog/Hive metastore has to track and the query planner has to evaluate every partition), which can paradoxically make queries slower despite 'more' partitioning. The right granularity is driven by actual query patterns and data volume per partition — a reasonable target is partitions large enough to contain at least ~128MB-1GB of compressed data each, and a partition column that matches how the data is actually filtered in practice (commonly date, sometimes date plus a secondary high-value dimension like region if query patterns justify it).

**Q: How would you approach reducing the overall AWS cost of a data platform without degrading SLAs?**

A: Start with usage-based analysis rather than guessing: use Cost Explorer/Cost and Usage Reports to identify the largest cost drivers (often EMR/Glue compute, Redshift cluster sizing, and S3 storage class misuse). Common concrete levers: right-size Redshift (check WLM queue utilization and consider Redshift Serverless or RA3 with managed storage if the cluster is over-provisioned for actual query load), move EMR task-node capacity to Spot instances, apply S3 Intelligent-Tiering or lifecycle rules to data that's rarely accessed after a certain age, fix the small-files/unpartitioned-table issues driving up Athena scan costs, and audit for orphaned resources (idle EMR clusters, unused EBS volumes, forgotten dev environments) that accumulate cost with zero business value. The SLA-protection piece is doing this incrementally with monitoring in place — e.g., testing Spot on task nodes only (not core), and validating query performance after a Redshift resize before declaring it safe — rather than making sweeping changes and hoping nothing breaks.

## 12. Scenario-Based Questions

Core approach for any scenario question: clarify requirements/constraints first (volume, latency, budget), state your proposed architecture end-to-end at a high level, then go deeper only where the interviewer probes — don't front-load every possible detail before confirming you're solving the right problem.

**Scenario: Design a nightly batch pipeline that ingests data from an on-premises SQL Server database into a data lake, transforms it, and makes it available for BI reporting.**

Approach: Use AWS DMS (Database Migration Service) or a Glue JDBC connection for extraction, landing raw extracts into an S3 bronze layer partitioned by extraction date; run a Glue ETL job to clean/conform the data into a silver layer (deduplication, type enforcement, schema validation); run a second Glue job or dbt-on-Athena/Redshift transformation to build gold-layer dimensional tables (facts/dimensions, potentially SCD Type 2 for slowly changing attributes); orchestrate the whole sequence with Step Functions or MWAA with defined retry/failure notification; and expose the gold layer to BI tools via Redshift (if interactive dashboard performance is required) or Athena (if a serverless, lower-cost option is acceptable for the query patterns).

**Scenario: A source system occasionally sends duplicate records due to at-least-once delivery in its upstream messaging system. How do you ensure your pipeline is idempotent?**

Approach: Design every write to be idempotent rather than trying to eliminate duplicates upstream: use a natural or composite business key (not an auto-generated ingestion-time ID) to identify a 'logical' record, and load data via UPSERT/MERGE semantics (Redshift's MERGE, or a Spark/Delta-style merge into a table format) keyed on that business key rather than blind INSERT/append — so reprocessing the same record twice overwrites rather than duplicates. For a pure append-only landing layer (bronze), duplicates are acceptable to keep since that layer is meant to be raw/immutable; the deduplication logic then lives explicitly in the bronze-to-silver transformation step (e.g., a window-function-based dedup keeping

the latest version per business key), which is also where you'd apply the same idempotency thinking to the transformation job itself (safe to re-run without side effects).

**Scenario: A downstream data consumer reports that a dashboard shows different numbers than yesterday, but no obvious pipeline failure occurred.**

Approach: Treat this as a data quality incident: first check whether the source data itself changed (a late-arriving correction, a backfill, or an upstream system reprocessing historical data), since that's a legitimate explanation distinct from a pipeline bug; check the pipeline's run history/logs for the relevant date range for silent partial failures (a job that 'succeeded' but processed fewer rows than expected due to a filter bug or a schema change silently dropping a column); compare row counts and key aggregate checksums between the current and previous run's output as a first diagnostic signal; and once resolved, evaluate whether a data quality check (e.g., a row-count or aggregate-value anomaly detection gate) should have caught this before it reached the dashboard, and add one if not.

**Scenario: You need to migrate a 50TB on-premises data warehouse to Redshift with minimal downtime for the business.**

Approach: Use AWS Schema Conversion Tool (SCT) to convert the source schema (handling any SQL dialect differences) and AWS DMS for the actual data migration, using DMS's full-load-plus-CDC mode — an initial full load of historical data while DMS simultaneously captures ongoing changes from the source, then a final short cutover window where remaining CDC changes are applied and the business switches its connection string to Redshift. For the Redshift side, load historical data via the COPY command from S3 (parallelized, far faster than row-by-row INSERT) with an initially reasonable distribution/sort key design based on known query patterns, and validate row counts/checksums between source and target before cutover. Minimizing downtime specifically comes from the CDC-based approach — the business keeps operating against the source system until the very final cutover, rather than requiring a multi-hour/day full outage for the whole 50TB to transfer.

**Scenario: Leadership wants near-real-time (under 5 minutes) visibility into sales data instead of the current next-day batch refresh.**

Approach: This is an architecture shift, not just a scheduling change: introduce a streaming ingestion path (Kinesis Data Streams or Firehose from the point-of-sale/order system) alongside the existing batch pipeline rather than trying to force the batch pipeline to run more frequently, since batch jobs typically have fixed overhead (cluster startup, full-table scans) that doesn't shrink linearly with a shorter interval. Stream processing (Kinesis Data Analytics/Flink, or Spark Structured Streaming on EMR) computes near-real-time aggregates into a fast-serving store (DynamoDB or a Redshift table updated via micro-batch MERGE), while the existing batch pipeline continues to run for full historical reconciliation and correcting any late-arriving/out-of-order data the streaming path approximated — a lambda-architecture-style pattern where speed and accuracy are handled by different layers rather than asking one pipeline to do both well.

**Scenario: A Glue crawler keeps misclassifying a table's schema because some files have an extra column that only appears intermittently.**

Approach: Rather than relying purely on crawler inference, define the table schema explicitly (via Glue Catalog API, Terraform, or CloudFormation) so it's authoritative and not re-derived from whatever sample of files the crawler happens to sample. If the extra column is a genuine, valid schema evolution (not bad data), configure the crawler's schema change policy to update the table to add the new column as nullable rather than creating a new versioned table or marking it deprecated, and set downstream jobs to explicitly select the columns they expect rather than using `SELECT *` so an unexpected extra column doesn't silently propagate. If the intermittent column is actually bad/malformed data from a misbehaving source, that's a data quality issue to catch and quarantine at ingestion rather than a schema problem to work around at the catalog level.

**Scenario: Your Redshift cluster's query queue is backing up during peak morning hours, delaying dashboard refreshes for business users.**

Approach: Check WLM queue configuration first — if all queries (short dashboard queries and long ETL/ad hoc queries) share one queue, a few long-running queries can starve short ones of slots; splitting into separate WLM queues (or using Redshift's automatic WLM / query priority feature) with dashboard queries in a high-priority, short-timeout queue and heavy ETL in a separate lower-priority queue is usually the first fix. If queue contention persists even after separating workloads, consider whether the ETL load window can shift earlier (before peak dashboard hours) or whether Redshift Serverless / elastic resize / a second reader-focused cluster (via data sharing) would isolate the reporting workload from the write/ETL workload entirely, since sharing one cluster for both is a common root cause of exactly this kind of peak-hour contention.

**Scenario: A compliance audit finds that several IAM roles used by data pipelines have far broader S3 permissions than they need.**

Approach: Use IAM Access Analyzer's policy generation feature (which examines actual CloudTrail access history) to see exactly which S3 actions and resources each role has genuinely used over a representative period, then tighten each role's policy to match observed usage rather than the broad wildcard permissions currently granted. Roll changes out gradually with monitoring (e.g., switch to the tightened policy in a non-production account first, or use IAM policy simulation before applying to production) since removing permissions blindly risks breaking a legitimate but infrequent access pattern (e.g., a monthly job) that wasn't captured in a short observation window. Going forward, I'd also recommend a periodic (e.g., quarterly) least-privilege review as a standing practice rather than something only caught reactively during an audit.

### 13. Behavioral Questions (STAR Prep)

Core concept: STAR = Situation (brief context), Task (what you specifically needed to do), Action (what you actually did — the bulk of the answer, in first person, technical enough to be credible), Result (the measurable/observable outcome, plus what you'd do differently or what changed afterward). Prepare stories in advance; don't improvise them cold.

- Tell me about a time a pipeline you built failed in production. What happened and how did you resolve it? — Prepare a specific example with a clear root cause (not just 'it

broke'), your diagnostic steps, the fix, and what monitoring/safeguard you added afterward so the same failure mode can't recur silently.

- Describe a time you had to optimize a slow-running pipeline or query. — Lead with the measurable before/after (e.g., 'reduced a nightly job from 3 hours to 40 minutes'), and be ready to explain the actual technical root cause you found, not just that you 'tuned it.'
- Tell me about a time you disagreed with a technical decision (e.g., a tool or architecture choice) and how it was resolved. — Show that you can advocate for a position with technical reasoning while still being collaborative about the outcome, whichever way it went.
- Describe a situation where you had to work with incomplete or poor-quality source data. — Focus on how you detected the quality issue, how you decided whether to fix it upstream vs. handle it defensively downstream, and how you communicated the limitation to consumers of the data.
- Tell me about a time you had to learn a new tool or technology quickly for a project. — Useful for showing you can ramp on unfamiliar parts of the AWS data stack quickly, which matters given how broad this ecosystem is.